

Databasesystems 24-hour exam

By Simon Holm - sihol24 - 16/12/2002

DM576 - Databasesystems

Teacher: Panagiotis Tampakis

Contents

1	Introduction	3
2	E/R Diagram and Relational Model	3
2.1	(a)	4
2.2	(b)	5
2.3	(c)	5
3	Part 2: Database Design	6
3.1	(a)	6
3.2	(b)	6
3.3	(c)	7
3.4	(d)	7
3.5	(e)	8
4	Relational algebra	9
4.1	(a)	9
4.2	(b)	10
5	Normalization	11
5.1	(a)	11
5.2	(b)	12
5.3	(c)	13
5.4	(d)	14
6	Indices	15
6.1	(a)	15
6.2	(b)	15

1 Introduction

This is the report for the 24 hour exam in DM576 - Databasesystems

2 E/R Diagram and Relational Model

Consider the following Database

- **User** having ID, Name, Address and Email.
- **Restaurant** having ID, Name, Address, Email and Phone Number.
- **CuisineType** having Type of Cuisine.
- **MenuItem** having ID, Name and Price.
- **Order** having ID, Time of Order, TotalAmount.

You should also consider the following: A user can order multiple items from a restaurant and a user can place an order at exactly one restaurant at a specific timestamp. We would also like to keep track of the quantity of ordered items. A restaurant can belong to multiple types of cuisine. Each menu item has an identifier that distinguishes it from all other items in the database. Furthermore, each menu item can either be a meal or a beverage. For each meal, we would like to keep track of whether it is a vegetarian or not and the number of calories. For each beverage, we would like to keep track of whether it is alcoholic or not and its volume. In case a beverage is alcoholic, we would also like to keep track of the alcohol percentage.

2.1 (a)

Create an E/R diagram capturing the objects and relationships described above. Describe all your design choices and constraints. Your assumptions cannot override the description above. Please use the notation for E/R diagrams introduced in the course book or from the slides. For each entity set, determine an appropriate key and underline it by making the appropriate assumptions, if necessary.

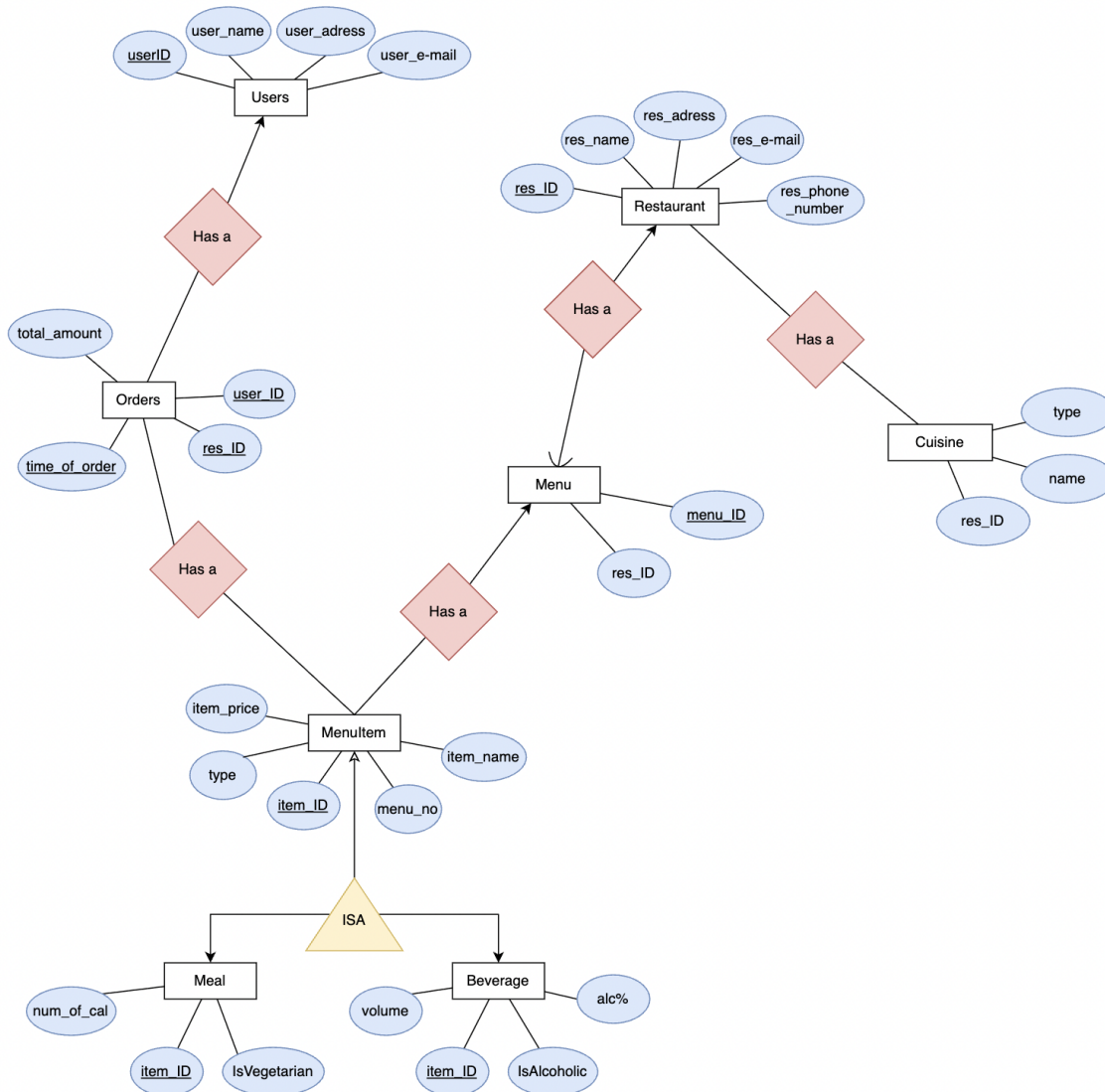


Figure 1: ER-Diagram for the Restaurant model

For the E/R-diagram above the `menu_item` has been split using an "ISA" to ensure that a `menu_item` can either be a `Meal` or a `Beverage`. Furthermore attributes such as `Volume` or `isVegan` has been made as requested in the assignment. The many to many relation between `Restaurant` and `Cuisine` to ensure that all restaurant can have multiple cuisines. The `Orders` is supposed to uphold the constraint a user can only make on order on a specific restaurant on a specific timestamp. Therefore it has 3 primary keys. The `Users` entity uses a "Has A" connection to the `Orders` entity. This is a one to many relation as A user can have many orders, but a order can only be placed by one user.

2.2 (b)

Convert the E/R diagram from (a) to a relational schema. Describe the process and justify the choices you have made.

- Users(user_ID, user_name, user_address, user_e-mail)
- Restaurant(res_ID, res_name, res_address, res_e-mail, res_phone_number)
- Orders(order_ID, total_amount, time_of_order, user_ID, res_ID)
- Menu(menu_ID, res_ID)
- Cuisine(type, name, res_ID)
- MenuItem(item_ID, item_name, item_price, type, menu_no)
- Meal(item_ID, num_of_cal, isVegetarian)
- Beverage(item_ID, volume, isAlcoholic, alc%)

The ER diagram has been written above as a relational schema, indicating attributes as primary keys with underscore.

2.3 (c)

Let us consider that a restaurant can belong to exactly one cuisine type and that the menu item identifier distinguishes a menu item only from all other items of the same restaurant. Modify the E/R diagram and the relational schema to accommodate these changes.

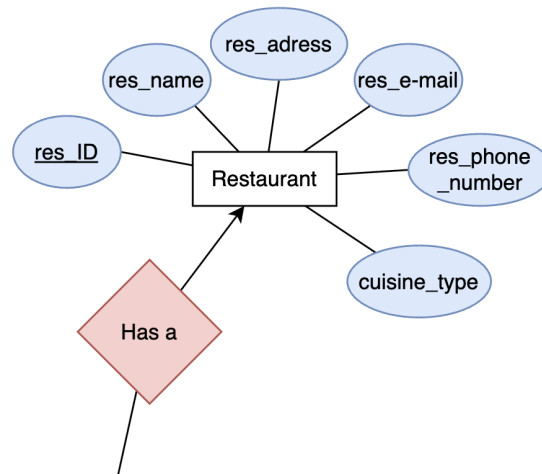


Figure 2: ER diagram with modification

The Cuisine entity has been removed and replaced with a single attribute, ensuring that a restaurant only has one type of cuisine.

- Users(user_ID, user_name, user_address, user_e-mail)
- Restaurant(res_ID, res_name, res_address, res_e-mail, res_phone_number, cuisine_type)
- Orders(order_ID, total_amount, time_of_order, user_ID, res_ID)
- Menu(menu_ID, res_ID)
- MenuItem(item_ID, item_name, item_price, type, menu_no)
- Meal(item_ID, num_of_cal, isVegetarian)
- Beverage(item_ID, volume, isAlcoholic, alc%)

This is also done in the relational schema.

3 Part 2: Database Design

Let us assume the following relational schema:

- Publisher(PublisherID, Name, Address, Email)
- Book(ISBN, PublisherID, Name, Description)
- Genre(ISBN, PublisherID, Name, Description)
- BookGenre(ISBN, GenreID)
- Author(AuthorID, Name, Address, Email)
- BookGenre(BookID, AuthorID, AuthorshipOrder)

The first table, **Publisher**, contains information about the publishers. Each publisher has a unique numeric **PublisherID**. It also has a name, an address and an email address. The second table, **Book**, contains information about the books. Each book has a unique ISBN, the id of the publisher, a name and a description. The third table, **Genre**, contains information about the available genres. Each genre has a unique numeric **GenreID**, a name and a description. The fourth table, **BookGenre**, contains information about the genres to which each book belongs to. The fifth table, **Author**, contains information about the authors. Each author has a unique numeric **AuthorID**, followed by a name, an address and an email address. The sixth table, **BookAuthor**, contains information about which authors wrote a book. The first column is the **BookID**, followed by the **AuthorID** and the order of authorship. A publisher can be registered in the database without having published a book yet. In the following tasks, you should write single SQL expressions. You are allowed to use subqueries, unless indicated otherwise.

3.1 (a)

Write an SQL **SELECT** query (without subqueries), which returns the number of books per genre. The result should consist of the genre name and number of books.

```
1  SELECT G.Name GenreName, COUNT(BG.ISBN) NumberOfBooks
2  FROM Genre G
3  JOIN BookGenre BG ON G.GenreID = BG.GenreID
4  GROUP BY G.Name;
```

The query above retrieves the number of books per genre by **JOINing** the **Genre** and **BookGenre** tables. It **GROUPs** the results by genre name and counts the number of books associated with each genre. To find how many books belong to each genre, the genre list is combined with the table that links books to genres. For each genre, the number of linked entries is counted. each one representing a book in that genre. The output then shows each genre name alongside the corresponding number of books.

3.2 (b)

Write an SQL **SELECT** query (without subqueries), which returns the name of the publishers that have published at most 10 books. The result should consist of the publisher name and number of books published.

```
1  SELECT P.Name PublisherName, COUNT(B.ISBN) NumberOfBooks
2  FROM Publisher P
3  JOIN Book B ON P.PublisherID = B.PublisherID
4  GROUP BY P.Name
5  HAVING COUNT(B.ISBN) <= 10;
```

The query above checks how many books each publisher has published. It **JOINs** publishers with books using their IDs, counts the books for each publisher, and shows only those publishers that have published 10 or fewer books. The result then includes the publisher's name and the number of books they've published.

3.3 (c)

Write an SQL SELECT query (without subqueries), which returns all authors that have not written a book that belongs to the genre “Mystery”. The result should consist of the AuthorID and the AuthorName.

```
1  SELECT A.AuthorID, A.Name
2  FROM Author A
3      LEFT JOIN BookAuthor BA ON A.AuthorID = BA.AuthorID
4      LEFT JOIN BookGenre BG ON BA.BookID = BG.ISBN
5      LEFT JOIN Genre G ON BG.GenreID = G.GenreID AND G.Name = 'Mystery'
6  WHERE G.GenreID IS NULL;
```

The query above gets a list of all authors and checks if any of their books are in the “Mystery” genre. It does this by linking authors to books, then linking those books to genres. The **LEFT JOIN** keeps all authors, even if they have no books in that genre. The **WHERE G.GenreID IS NULL** part filters out anyone who has written a “Mystery” book, so the result only shows authors who haven’t. **Note:** This query assumes that if there’s no match for “Mystery” in the Genre table, the author hasn’t written any books in that genre. But if the data is messy — like missing genre links, typos in the genre name, or NULL values — the result might be off. So, it’s important the data is complete and consistent for this to work correctly.

3.4 (d)

Write an SQL SELECT query, which returns the authors with the most and least written books. If there is a “tie” in the most (or least) written books, then all of them should appear in the result. The result should consist of the AuthorID and the AuthorName.

To return authors with the most and least number of books written, including ties, the optimal approach is to use subqueries that first count the number of books per author, then filter to only those with the maximum or minimum count.

```
1  SELECT AuthorID, Name
2  FROM (
3      SELECT A.AuthorID, A.Name, COUNT(BA.BookID) AS BookCount
4      FROM Author A
5      JOIN BookAuthor BA ON A.AuthorID = BA.AuthorID
6      GROUP BY A.AuthorID, A.Name
7  ) AS AuthorBookCounts
8  WHERE BookCount = (
9      -- select the minimum
10     SELECT MAX(CountPerAuthor)
11     FROM (
12         SELECT COUNT(BA.BookID) AS CountPerAuthor
13         FROM BookAuthor BA
14         GROUP BY BA.AuthorID
15     ) AS Counts
16 )
17 OR BookCount = (
18     -- select the maximum
19     SELECT MIN(CountPerAuthor)
20     FROM (
21         SELECT COUNT(BA.BookID) AS CountPerAuthor
22         FROM BookAuthor BA
23         GROUP BY BA.AuthorID
24     ) AS Counts
25 );
```

This query first builds a list showing how many books each author has written. Then it checks for the highest and lowest number of books written. Any author with one of those counts (including ties) is returned, showing their ID and name.

3.5 (e)

Write an **SQL SELECT** query, which returns for each publisher, the number of publishers that have published more books. The result should consist of the **PublisherID**, the publisher Name and the number of publishers that have published more books.

To get, for each publisher, the number of publishers who have published more books, the query involves: Counting books per publisher, comparing each publisher's book count to others and counting how many publishers have a strictly higher book count.

```
1      -- counts how many books each publisher has published
2      WITH PublisherBookCounts AS (
3          SELECT P.PublisherID, P.Name, COUNT(B.ISBN) AS BookCount
4          FROM Publisher P
5          LEFT JOIN Book B ON P.PublisherID = B.PublisherID
6          GROUP BY P.PublisherID, P.Name
7      )
8
9      SELECT
10         P1.PublisherID,
11         P1.Name,
12         COUNT(P2.PublisherID) NumPublishersWithMoreBooks
13     FROM
14         PublisherBookCounts P1
15     LEFT JOIN
16         PublisherBookCounts P2 ON P2.BookCount > P1.BookCount
17     GROUP BY
18         P1.PublisherID, P1.Name
19     ORDER BY
20         P1.PublisherID;
```

The query above firstly, counts how many books each publisher has published (including zero). Then, for each publisher, it counts how many other publishers have published more books. The final result shows each publisher's ID, name, and how many publishers have a strictly higher book count.

4 Relational algebra

These tasks concern extended relational algebra on bags. Consider three relations, $R(A, B, C)$, $S(A, B, D)$ and $T(B, C, D)$.

4.1 (a)

For each of the following expressions, give the equivalent sql query.

- $\sigma_{B>C}(T) \times R$

```
1      SELECT *
2      FROM T
3      CROSS JOIN R
4      WHERE T.B > T.C;
```

The query above **SELECTS**(σ) from relation T where $B > C$ and then cross joins with R .

- $\Pi_{B,D}(S) - \Pi_{C,A}(R)$

```
1      SELECT B,D
2      FROM S
3
4      EXCEPT
5
6      SELECT C,A
7      FROM R;
```

The query above projects(Π) B and D from relation S specifically without C and A from relations R .

- $(T \bowtie S) \cap (R \bowtie S)$ Since outerjoin needs a condition, i assume they use B since it is present in all relations.

```
1      SELECT *
2      FROM T
3           LEFT OUTER JOIN S ON B
4
5      INTERSECT
6
7      SELECT *
8      FROM R
9           RIGHT OUTER JOIN S ON B;
```

The query above **INTERSECT**($\cap$) while left and right joining.

- $\sigma_{C>D}(R \times S)$

```
1      SELECT *
2      FROM R CROSS JOIN S
3      WHERE C > D;
```

- $\gamma_{B,C,\text{sum}(A)}(T \bowtie S)$ Since outerjoin needs a condition, i assume they use B since it is present in all relations.

```

1      SELECT B, C, SUM(A) AS TotalA
2      FROM T LEFT OUTER JOIN S ON T.B = S.B
3      GROUP BY B, C;
```

The query above uses `GROUP BY`(γ) to group by A, B and $SUM(A)$

4.2 (b)

For each of the following SQL statements, write an equivalent expression of extended relational algebra:

- `SELECT R.A, S.B, C, D FROM S RIGHT OUTER JOIN R ON C = D AND R.A > S.A;`

$$\Pi_{(R.A, S.B, C, D)}(S \bowtie_{(C=D \text{ AND } R.A > S.A)} R)$$

- `SELECT DISTINCT B FROM T NATURAL JOIN R WHERE C = 3 OR D = 3;`

$$\pi_B(\sigma_{C=3 \vee D=3}(T \bowtie R))$$

- `SELECT S.A, SUM(T.C) FROM S, T WHERE S.B = T.B GROUP BY S.A;`

$$\gamma_{S.A, \text{sum}(T.C)}(\sigma_{S.B=T.B}(S \times T))$$

5 Normalization

Consider the following relation:

$R(\text{UserID}, \text{UserName}, \text{UserAddress}, \text{RestaurantID}, \text{RestaurantName}, \text{RestaurantAddress}, \text{CuisineType}, \text{OrderTime}, \text{ItemID}, \text{ItemName}, \text{ItemQuantity}, \text{ItemPrice})$.

Firstly, let's define all attributes to letters, this makes them much easier to work with:

- $A = \text{UserID}$
- $B = \text{UserName}$
- $C = \text{UserAddress}$
- $D = \text{RestaurantID}$
- $E = \text{RestaurantName}$
- $F = \text{RestaurantAddress}$
- $G = \text{CuisineType}$
- $H = \text{OrderTime}$
- $I = \text{ItemID}$
- $J = \text{ItemName}$
- $K = \text{ItemQuantity}$
- $L = \text{ItemPrice}$

5.1 (a)

List all of the functional dependencies of R based on the description of Section 1.

Note: It is assumed that one restaurant can only have one cuisine.

- $A \rightarrow BC$
- $D \rightarrow EFG$
- $ADH \rightarrow K$
- $I \rightarrow JL$

Note: these functional dependencies are assumed, based on the work done in Section 1

5.2 (b)

List all keys of R by computing the closure of all necessary combinations of attributes and argue why there can be no other keys.

Step 1: Consider candidate keys starting from minimal attribute sets

- Compute $(A, D, H, I)^+$ (closure of A, D, H, I):

$$\{A, D, H, I\}$$

$$\xrightarrow{A \rightarrow BC} \{A, B, C, D, H, I\}$$

$$\xrightarrow{D \rightarrow EFG} \{A, B, C, D, E, F, G, H, I\}$$

$$\xrightarrow{I \rightarrow JL} \{A, B, C, D, E, F, G, H, I, J, L\}$$

$$\xrightarrow{ADH \rightarrow K} \{A, B, C, D, E, F, G, H, I, J, K, L\}$$

This covers all attributes, so (A, D, H, I) is a superkey.

- Can any smaller combination be a key?
 - $(A, D, I)^+$ misses H , so no.
 - $(A, H, I)^+$ misses D , so no.
 - $(D, H, I)^+$ misses A , so no.
 - $(A, D, H)^+$ misses I , so no.
- Can A alone be a key? No, it only determines B, C .
- Can D alone be a key? No, it only determines E, F, G .
- Can I alone be a key? No, it only determines J, L .
- Can H alone be a key? No functional dependencies start with H alone.

The only candidate key for relation R is $\boxed{\{A, D, H, I\}}$ since it functionally determines all attributes in the relation, and no subset of these attributes determines all attributes.

5.3 (c)

Analyze whether R is in 3NF, if it is, show that there are no 3NF violations. If not, decompose it into 3NF using the Synthesis Algorithm for 3NF Schemas.

Check 3NF Violations

By the Database Book definition:

A relation schema is in **Third Normal Form (3NF)** if, for every non-trivial functional dependency $X \rightarrow A$, at least one of the following holds:

- X is a **superkey**, or
- A is a **prime attribute** (i.e., part of some candidate key)

This is done on the 4 FDs

- $A \rightarrow BC$
 A is not a superkey, B, C are not prime $\Rightarrow$ **violation**
- $D \rightarrow EFG$
 D is not a superkey, E, F, G are not prime $\Rightarrow$ **violation**
- $ADH \rightarrow K$
 ADH is not a superkey (missing I), K not prime $\Rightarrow$ **violation**
- $I \rightarrow JL$
 I is not a superkey, J, L not prime $\Rightarrow$ **violation**
- Relation R is **not** in 3NF.

For the decomposition, the **Synthesis Algorithm** is mentioned in the Database Book by:

Perform the following steps:

1. Find a minimal basis for the set of functional dependencies F , denote it by G .
2. For each functional dependency $X \rightarrow A$ in G , create a relation schema XA (i.e., with attributes $X \cup \{A\}$) as part of the decomposition.
3. If none of the relation schemas created in Step 2 contains a superkey for the original relation R , then add another relation whose schema is a key for R .

$$\text{Minimal Basis} \rightarrow \left\{ \begin{array}{l} A \rightarrow B \\ A \rightarrow C \\ D \rightarrow E \\ D \rightarrow F \\ D \rightarrow G \\ ADH \rightarrow K \\ I \rightarrow J \\ I \rightarrow L \end{array} \right.$$

These can be used to make relations

- $R_1(A, B, C)$ from $A \rightarrow B, A \rightarrow C$
- $R_2(D, E, F, G)$ from $D \rightarrow E, D \rightarrow F, D \rightarrow G$
- $R_3(A, D, H, K)$ from $ADH \rightarrow K$
- $R_4(I, J, L)$ from $I \rightarrow J, I \rightarrow L$
- $R_5(A, D, H, I)$ to include the candidate key and ensure lossless join

Each of the relations above is in 3NF. This can be checked by the rule from the book earlier.

5.4 (d)

What would change if we considered BCNF instead of 3NF? If R is in BCNF, show that there are no BCNF violations. If not, decompose it into BCNF using the BCNF Decomposition Algorithm.

For BCNF it is no longer enough for a relation to have prime attributes. They now strictly all have to be super-keys

BCNF Violation Check

- $A \rightarrow BC$
 A is not a superkey $\Rightarrow$ **violation**
- $D \rightarrow EFG$
 D is not a superkey $\Rightarrow$ **violation**
- $ADH \rightarrow K$
 ADH is not a superkey (missing I) $\Rightarrow$ **violation**
- $I \rightarrow JL$
 I is not a superkey $\Rightarrow$ **violation**

Conclusion: Relation R is **not** in BCNF.

BCNF Decomposition Steps

1. Decompose on $A \rightarrow BC$:
 - $R_1 = \{A, B, C\}$
 - $R_2 = \{A, D, E, F, G, H, I, J, K, L\}$
2. In R_2 , decompose on $D \rightarrow EFG$:
 - $R_3 = \{D, E, F, G\}$
 - $R_4 = \{A, D, H, I, J, K, L\}$
3. In R_4 , decompose on $I \rightarrow JL$:
 - $R_5 = \{I, J, L\}$
 - $R_6 = \{A, D, H, I, K\}$
4. In R_6 , decompose on $ADH \rightarrow K$:
 - $R_7 = \{A, D, H, K\}$
 - $R_8 = \{A, D, H, I\}$

Final BCNF Relations

- $R_1(A, B, C) \Rightarrow R_1(\text{UserID}, \text{UserName}, \text{UserAddress})$
- $R_3(D, E, F, G) \Rightarrow R_3(\text{RestaurantID}, \text{RestaurantName}, \text{RestaurantAddress}, \text{CuisineType})$
- $R_5(I, J, L) \Rightarrow R_5(\text{ItemPrice}, \text{ItemName}, \text{ItemPrice})$
- $R_7(A, D, H, K) \Rightarrow R_7(\text{UserID}, \text{RestaurantID}, \text{OrderTime}, \text{ItemQuantity})$
- $R_8(A, D, H, I) \Rightarrow R_8(\text{UserID}, \text{RestaurantID}, \text{OrderTime}, \text{ItemPrice})$

Each of these relations is in BCNF, as all remaining functional dependencies have a superkey as the left-hand side.

6 Indices

Consider that the schemas presented in Section 2 get implemented in PostgreSQL.

6.1 (a)

Write CREATE statements for indices based on the primary keys (use the PostgreSQL syntax).

```
1      CREATE INDEX idx_publisher_publisherid
2          ON Publisher(PublisherID);
3
4      CREATE INDEX idx_book_isbn
5          ON Book(ISBN);
6
7      CREATE INDEX idx_genre_genreid
8          ON Genre(GenreID);
9
10     CREATE INDEX idx_bookgenre_isbn_genreid
11         ON BookGenre(ISBN, GenreID);
12
13     CREATE INDEX idx_author_authorid
14         ON Author(AuthorID);
15
16     CREATE INDEX idx_bookauthor_bookid_authorid
17         ON BookAuthor(BookID, AuthorID);
```

6.2 (b)

For queries (a), (c) and (e) of Section 2, investigate if you can speed up their execution by building some index. If so, write the CREATE INDEX statements. In any case, argue about your choice and justify your decisions.

As mentioned in the Database Book. Indexes are especially useful when dealing with large amounts of data, and when using either **WHERE** clauses or **JOIN** clauses. In the case of all 3 queries a, b and c, if these were to handle large amounts of data, using indexes would most definitely improve the time of execution.